

Lecture #1

Instructor: Yunseok Choi
(ys.choi@skku.edu)

What is Machine Learning?

Machine Learning

■ 머신러닝

- 기계가 학습을 잘하도록 하는 연구 분야
- 인공지능 연구의 한 분야로 최근 딥러닝을 통해 빠르게 발전하고 있음

Artificial Intelligence

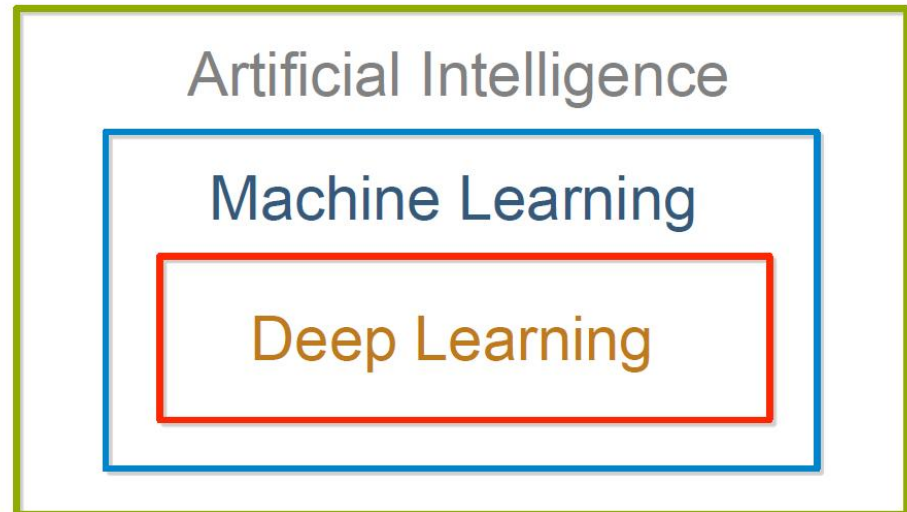
Any technique which enables computers to mimic human behavior.

Machine Learning

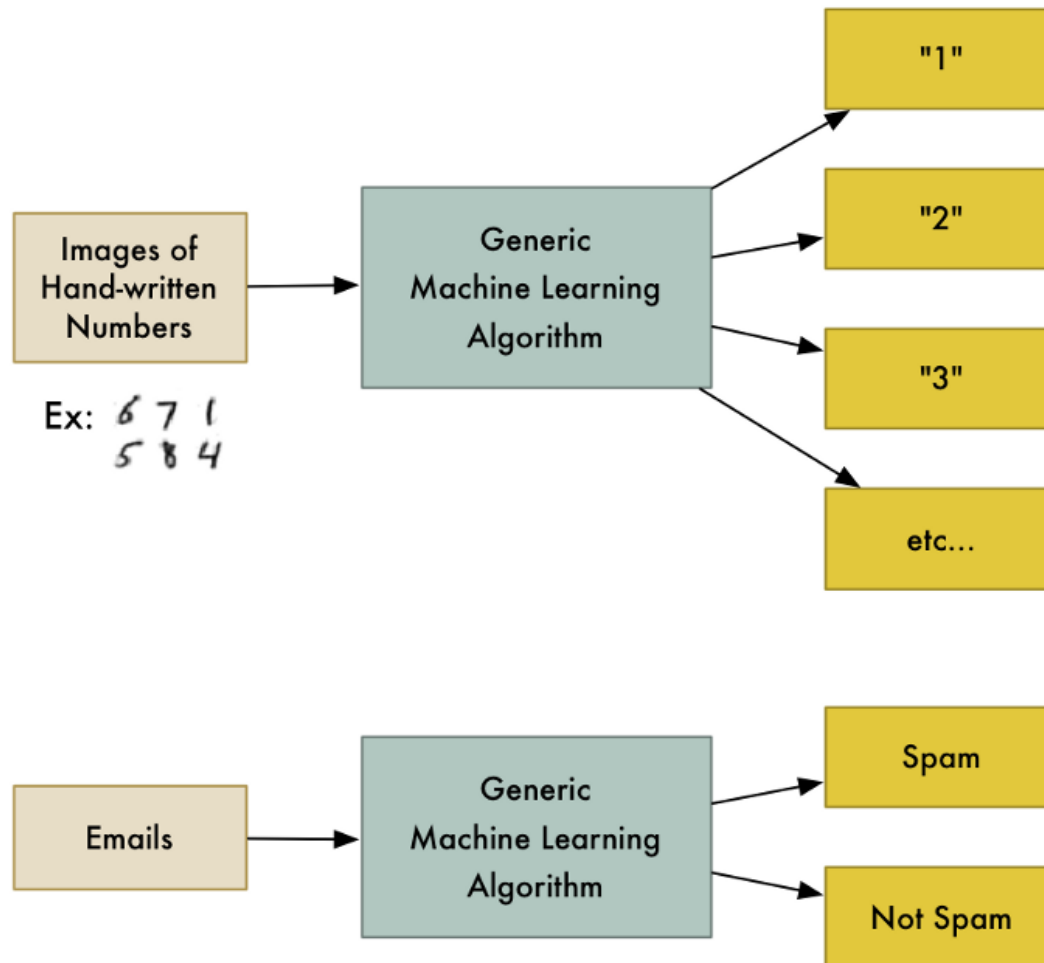
Subset of AI techniques which use statistical methods to enable machines to improve with experiences.

Deep Learning

Subset of ML which make the computation of multi-layer neural networks feasible.



Machine Learning 예



Classical Definition

■ Well-Posed Learning Problems

- A computer program is said to *learn* from *experience* E with respect to some class of *tasks* T and *performance* measure P, if its performance at tasks in T, as measured by P, improves with experience E.
- 환경(Environment, E)과의 상호작용을 통해서 축적되는 경험적인 데이터(Data, D)를 바탕으로 지식 즉 모델(Model, M)을 자동으로 구축하고 스스로 성능(Performance, P)을 향상하는 시스템
 - (Mitchell, 1997)

Examples of ML Problems

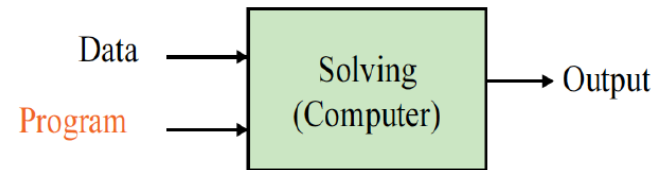
- A checkers learning problem
 - Task T: playing checkers
 - Performance measure P: percent of games won against opponents
 - Training experience E: playing practice games against itself
- A handwriting recognition learning problem
 - Task T: recognizing and classifying handwritten words within images
 - Performance measure P: percent of words correctly classified
 - Training experience E: a database of handwritten words with given classifications
- A robot driving learning problem
 - Task T: driving on public four-lane highways using vision sensors
 - Performance measure P: average distance traveled before an error (as judged by human overseer)
 - Training experience E: a sequence of images and steering commands recorded while observing a human driver

프로그래밍 방식과의 차이점

■ 일반적인 컴퓨터 프로그램

- 사람이 알고리즘 설계 및 코딩
- 주어진 문제(데이터)에 대한 답 출력

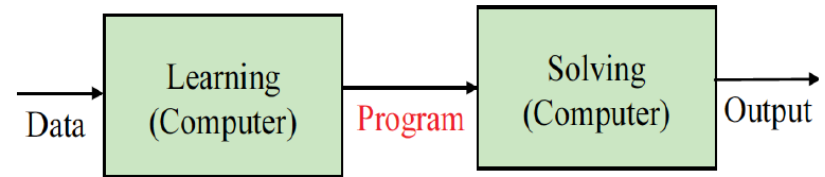
Human Programming



■ 머신러닝 프로그램

- 사람이 코딩
- 기계가 알고리즘을 자동 프로그래밍
- 데이터에 대한 프로그램을 출력

Automatic Programming (Deep Learning)



Machine Learning의 필요성

- 머신러닝이 필요한 문제
 - 명시적 문제해결 지식의 부재(알고리즘 부재)
 - 프로그래밍이 어려운 문제(예: 음성인식)
 - 지속적으로 변화하는 문제(예: 자율이동로봇)
- 머신러닝 더욱 중요해지는 이유
 - 빅데이터의 존재(학습에 필요)
 - 컴퓨팅 성능의 향상(고난도 학습이 가능)
 - 새로운 비즈니스 가치 창출(신산업/서비스)

Machine Learning 활용 분야

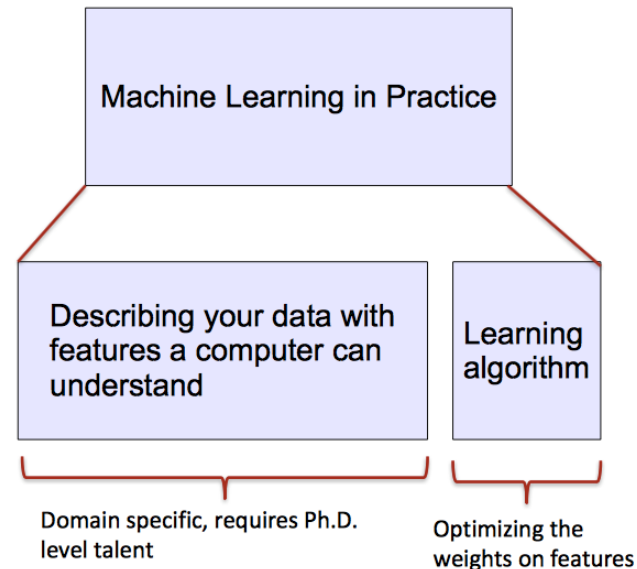
- 데이터마이닝, 텍스트마이닝, 웹마이닝
- 자연어처리, 음성인식, 번역, 챗봇
- 컴퓨터비전, 물체인식, 네비게이션
- 로봇틱스, 휴머노이드 로봇, 모바일 로봇, 자율주행
- 로봇어드바이저, 의약학, 바이오인포매틱스
- 비즈니스 분석, 전자상거래, 신용카드 도용 검출, 검색, 추천시스템

Machine Learning 역사



ML VS. DL

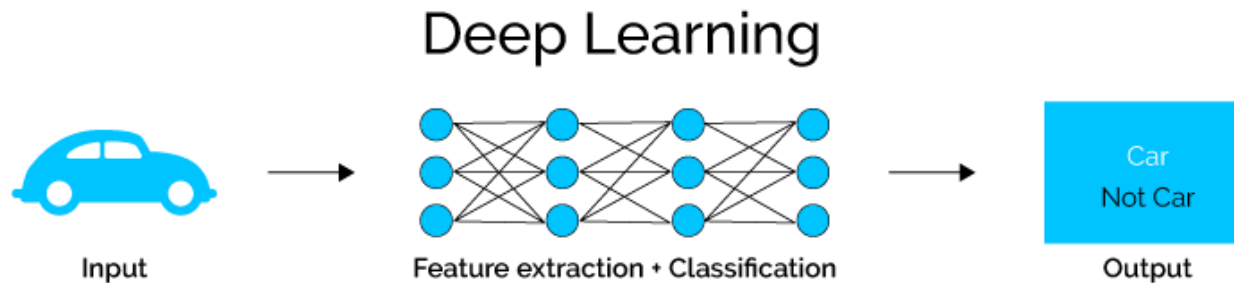
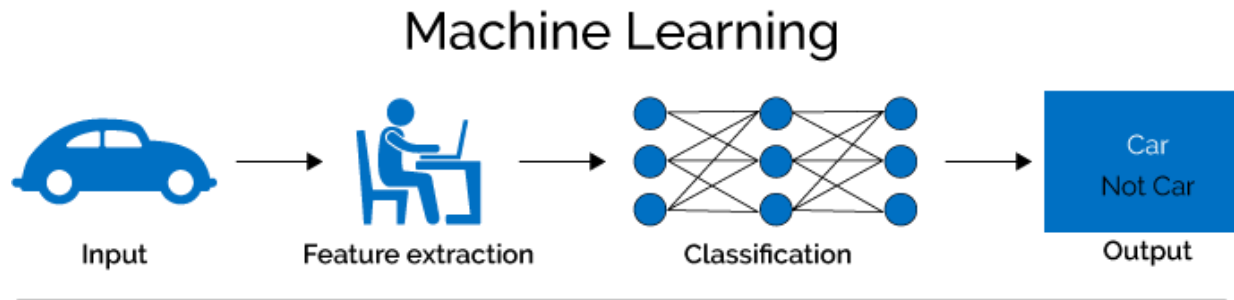
- Most machine learning methods work well
 - Due to human-designed representations and input features
- ML (somewhat) becomes optimizing weights to best make a final prediction



ML VS. DL

- Deep learning

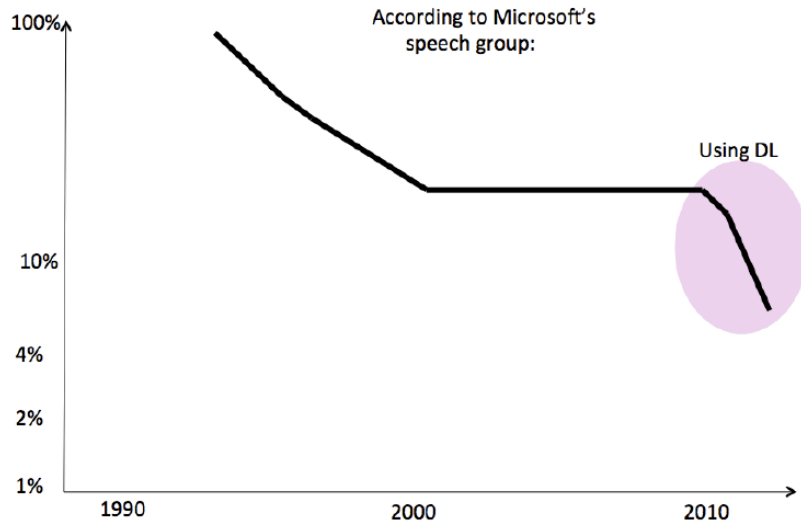
- A part of a broader family of machine learning methods based on learning data representations, as opposed to task-specific algorithm



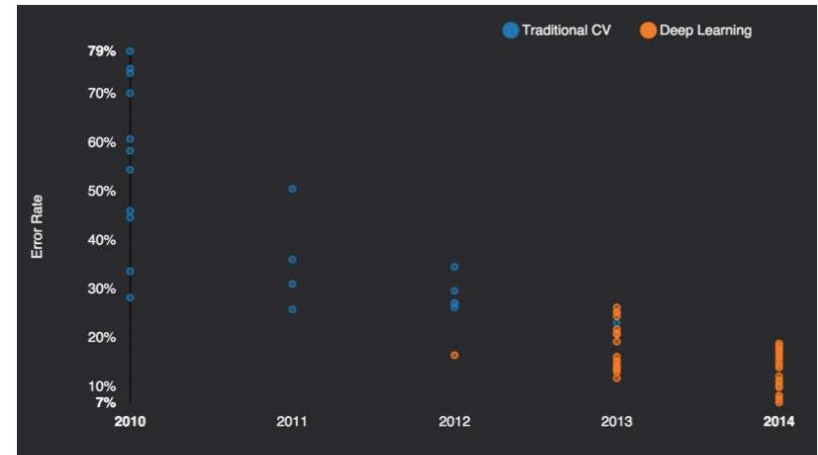
Deep Learning

- ML: manually designed features are often over-specified, incomplete, and take a long time to design and validate
- DL provides a very flexible, (almost?) universal, learnable framework for representing visual and linguistic information
- DL utilizes a large amount of training data
 - Pros and cons?
- From 2010, DL has started to outperform other ML techniques, first in speech and vision, then NLP

Deep Learning



Deep Learning in Speech Recognition

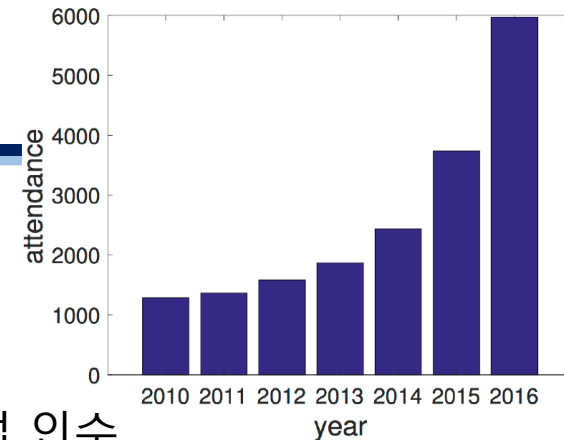


ImageNet: The "computer vision World Cup"

- Big improvements in recent years in NLP and computer vision!
 - DL + Big Data (e.g., ImageNet) + Computing Power

최근 산업 동향

NIPS Attendance 2016



■ 머신러닝에 대한 기업들의 투자 증가

- NIPS 학회의 참가자수 급증
- 글로벌 IT 기업들의 인공지능 연구소 설립 및 스타트업 인수
- Google의 8개의 로보틱스 회사 인수 및 인공지능 스타트업 투자와 딥러닝 연구자들 채용, DeepMind를 7000억원에 인수
- 중국의 바이두, 스탠포드의 Andrew Ng 교수를 영입해 실리콘밸리에 딥러닝 연구소 설립
- Facebook, Facebook AI Research 인공지능연구소 설립
- 테슬라, 인공지능회사 OpenAI 설립 공표
- 네이버, Naver Labs를 분사하여 인공지능 연구 사업화
- 삼성전자, 가전 제품과 스마트폰 등에 인공지능 기술 도입한 신제품 개발
- SKT, 음성기반 인공지능 스피커 NUGU 출시
- LG 전자, 가전제품 박람회 CES에서 다양한 형태의 인공지능 로봇 제품

Machine Learning Basic

왜 기계가 배워야 할까?

- 모든 상황을 프로그래밍 할 수 없다.
= 모든 상황에 대한 규칙을 다 만들어 놓을 수 없다.
- 어떤 일은 알고리즘으로 설명하기 어렵다.
예) 운전

머신 러닝 Machine Learning

"A computer program is said to learn from
experience E with respect to some class of
tasks T and performance measure P "

Mitchell 1997

Task T

- 기계에게 시키고 싶은 일들?

- 분류
- 예측
- 추천
- 식별
- 탐지
- 그룹핑
- ...

Performance P

- Accuracy
- Precision = 암이라고 진단한 케이스 중 진짜 암인 경우의 비율
- Recall = 진짜 암 환자 중 정말 암이라고 정확하게 진단한 비율
- F-measure
- ROC curve
- ...

precision: TP/cancer diagnoses

		Diagnosis	
		No cancer	Cancer
True state	No cancer	TN	FP
	Cancer	FN	TP

recall: TP/cancer true states

Experience E

경험은 결국 데이터로 ...

Action	Wind	Temp	Outlook	Humidity
Play (P)	Weak	Hot	Sunny	High
Play	Strong	Hot	Sunny	High
Stay (S)	Weak	Hot	Rain	High
Play	Weak	Mid	Overcast	High
Stay	Strong	Cold	Rain	Normal
Play	Weak	Cold	Overcast	Normal
Stay	Strong	Cold	Rain	Normal
Play	Weak	Mid	Sunny	Normal
Play	Weak	Cold	Sunny	Normal
Play	Strong	Mid	Overcast	Normal
Stay	Weak	Mid	Sunny	High
Stay	Strong	Mid	Rain	High
Play	Weak	Hot	Overcast	Normal
Play	Weak	Cold	Rain	High

	Feature vector (x)	Label (y)
#1	0.45, 0.36, 0.67 ...	Play
#2	0.75, 0.96, 0.87 ...	Stay
...	...	...

Experience E

- 기계에게 어떻게 경험을 만들어 줄 수 있을까?

→ 학습 알고리즘

- 경험시키는 방법은

데이터에 따라서도 다를 것이고

가르치는 방식에 따라서도 다를 것이다.

가르치는 방식

지도학습

Supervised Learning

비지도학습

Unsupervised Learning

강화학습

Reinforcement Learning

비지도학습 Unsupervised learning

- 기계에게 입력 데이터(상황)만 주고 기대하는 **출력(결과)는 알려주지 않는(못하는) 학습 방법**
 - 이러한 데이터를 **Unlabeled data**라 한다.
- 데이터 자체에서 기계가 숨겨진 구조와 패턴을 찾으려 함
 - 예) 마케팅 활동을 위해 유사한 온라인 행동 패턴을 지닌 고객을 그룹화
 - 그룹을 처음부터 정의되지 않고 기계가 그룹을 만들어 간다.

지도학습 Supervised learning

- 기계에게 입력 데이터(상황)와 출력(결과) 데이터를 함께 주며 학습 시키는 방법
 - 이러한 데이터를 Labeled data라 한다.
- 입력과 결과를 매핑하는 룰을 찾는 것이 학습 목표
 - 예) 얼굴 인식, 음성 인식, 상품 추천, 영화 추천, 영업 실적 전망 등

지도학습 Supervised learning

- 지도학습은 회귀와 분류로 더 세분화 가능
 - Regression Classification
- 회귀 : 연속성을 갖는 응답 변수를 학습하고 예측
 - 집 값 예측
- 분류 : 클래스(카테고리) Label을 찾는 것
 - 긍정/부정 같은 감성 분석
 - 채무 불이행 예측

준지도학습 Semi-supervised learning

- (소량의) Labeled data와 (대량의) Unlabeled data를 학습에 함께 사용하는 방법
- 목표는 대부분 지도학습과 동일
- 보통 Labeled data를 만드는데 Cost가 많이 들기 때문에 사용

* Labeled data만 써서 지도학습을 할 때보다 Unlabeled data를 함께 사용했을 때 성능이 좋아지기도 함.

강화학습 Reinforcement Learning

- 어떤 Agent (혹은 시스템)이 존재
- 주어진 목표가 있고 Agent는 목표를 이루기 위한 일련의 행동을 여러 번 수행해 가며 가장 목표를 잘 달성할 수 있는 행동 규칙을 찾아감
 - 학습데이터가 어떤 상황에서 어떤 행동을 할지에 대한 피드백 제공
- 예) 게임, 자율주행차, 알파고

Machine Learning Process

머신러닝 과정

단계	내용	담당자
문제 이해	어떤 문제를 풀어야 하는지 파악	도메인 전문가
데이터 이해 (탐색)	문제를 위해 사용할 수 있는 데이터가 어떤 것들이 있고 어떤 타입이고 어떤 특성이 있는지 파악	도메인 전문가 + 기술 전문가
데이터 준비 (전처리)	학습 데이터와 테스트 데이터 생성을 목표로 불완전한 데이터 처리와 특성 정의	기술 전문가
모델링	머신러닝 모델 생성	기술 전문가
평가	생성된 모델 성능 평가	기술 전문가
배포	실제 환경에 모델을 적용하여 시스템 화	서비스 운영/관리자

Feature Engineering

- 결측값 처리 : 결측값(입력되지 않은 값, NULL)이 있을 때 이를 제외하고 사용하거나, 다른 값으로 대체하는 방법
 - 대체 시에는 특성의 Mean, Median, Mode 등을 사용할 수 있다.
- 원 핫 인코딩 : 범주형 특성의 인코딩에 더미 변수를 사용
 - one-hot encoding

	Is_africa	Is_asia	Is_europe	Is_south_america	Is_north_america
Africa	True	False	False	False	False
Asia	False	True	False	False	False
Europe	False	False	True	False	False
South America	False	False	False	True	False
North America	False	False	False	False	True
Other	False	False	False	False	False

Feature Engineering

- 정규화 : 특성 값의 범위(데이터 단위)를 조정
 - 표준 정규화 (Standardization)
 - 데이터 값에서 특성의 평균을 빼고 표준편차로 나누어 정규화
 - 특성값이 정규 분포를 따르지 않는 경우 중앙값을 빼고 IQR로 나눈다.
(IQR : Inter-Quartile Range로 25%되는 값과 75%되는 값 사이 범위)
 - 스케일링 (Scaling)
 - 특정 범위에 있는 특성의 스케일을 0과 1사이로 조정하는 방법이 많이 사용
- 다항형 특성 : 기존 특성들을 조합해 새로운 다항 특성을 생성
 - 특성 A, B가 있으면 $A \times B$ (Interaction)을 새로운 특성을 추가

Feature Engineering

■ 파워 변환 (Power Transformation)

- 데이터가 정규 분포를 따르면 사용할 수 있는 확률/통계 기법들이 많아진다.
- 지수적으로 증가하는 데이터의 경우 Log 를 씌워서 정규 분포를 따르는 데이터를 만들기도 한다.

■ 비닝 (Binning)

- 특성들의 값을 여러 개의 bin에 할당해서 분리 시켜 놓는 것이 좋을 때가 있다.
- 예) 강수량 데이터 → 비가 왔는지 안 왔는지(예 or 아니오)로 단순화
- 정보 손실은 있을 수 있지만 특성 공간을 줄여서 오버피팅의 가능성을 줄인다.

모델의 조합 Ensemble Method

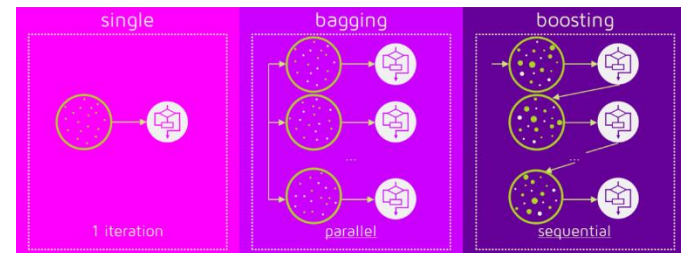
- 여러 모델을 독립적으로 학습 시킨 다음에
각 모델이 만들어 낸 결과를 조합하여 최종 결과를 생성
 - 배깅 (Bagging)
 - 부스팅 (Boosting)
 - 스택킹 (Stacking)
 - 블렌딩 (Blending)
 - 보팅 (Voting) 및 평균화 (Averaging)

배깅 Bagging

- 부트스트랩 집계(Bootstrap Aggregating)이라고도 함
- 하나의 학습 알고리즘이지만 여러 다른 데이터로 학습된 결과로 여러 모델을 만들고 이들의 출력 결과를 조합
- 단계
 - Replace 가능한 샘플링 방법으로 원본 학습 데이터에서 새로운 학습 데이터를 생성
 - 만들어진 학습 데이터 각각에 모델을 학습
 - 테스트 데이터에 대해서 학습된 모델들이 만들어낸 결과를 평균화 또는 최다 득표 보팅을 통해 최종 결과 생성

부스팅 Boosting

- 학습 모델을 순차적으로 연결하여 최종 결과를 내는 방법
- 한 모델에서 먼저 결과를 내고, 이 결과를 입력으로 다음 모델이 이어서 결과를 내는 방식
- 예) 이미지에서 얼굴 영역 찾기
 - 얼굴이 아닌 영역이 보통 더 넓기 때문에 찾기 쉽다.
 - 얼굴이 아닌 영역을 가리는 모델을 학습 시키고 순차적으로 실행
 - 한번 실행할 수록 얼굴이 아닌 영역을 제거하여 점점 입력 이미지가 작아지고 이는 특성 공간의 축소로 이어진다.



기타 다른 모델 조합 방법

- 스택킹(Stacking), 블렌딩(Blending)

- 부스팅처럼 학습 모델이 순차적으로 실행
- 하지만 실행 단계 별로 학습 모델의 역할과 기능을 다르게 설계

- 보팅과 평균화

- 학습 알고리즘을 다르게 하거나, 특성 집합을 다르게 입력해 여러 학습 모델을 만들고 각 모델의 결과를 보팅이나 평균화 방법을 통해 조합함

Overfitting

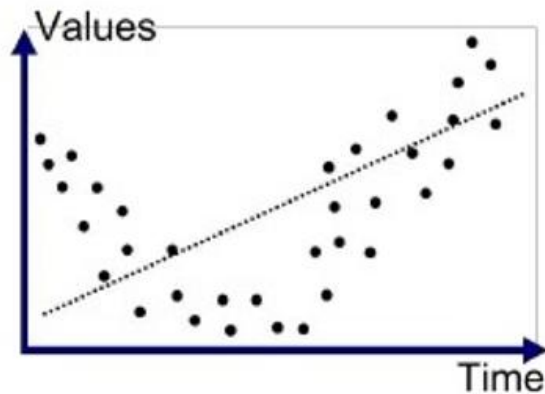
오버피팅 Overfitting

- 원리나 패턴을 찾지 않고 단순 암기하는 것은 **오버피팅**을 야기
 - ※ 오버피팅 : 기계학습 모델이 주어진 데이터에만 너무 특화되어 이전에 보지 못한 새로운 데이터에 대해 성능이 떨어지는 현상
- 이런 상황을 **Bias**와 **Variance**로 표현하기 도함
 - Bias는 가정(추정)이 얼마나 편향되어 있는가? (잘못되었는가?)
 - Variance는 입력 데이터의 변동에 따른 추정값 변화
- 오버피팅은 **Low Bias, High Variance**
 - 학습 데이터의 모든 샘플에 대해 편향 없이 잘 추정
 - 그러다 보니 학습 데이터의 작은 변화(노이즈 일수도 있는)에도 민감하게 반응하기 때문에 추정값 변화 폭 큼
 - 학습했던 데이터와 새로운 데이터 사이에 작은 차이만 있어도 추정값이 들쭉 날쭉
→ 일반화 성능 X

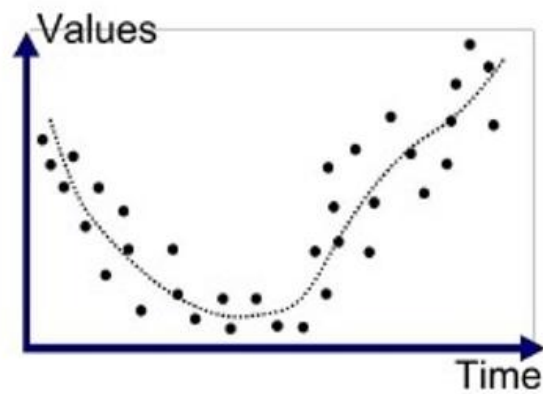
언더피팅 Underfitting

- 오버피팅의 반대!
- 학습 데이터셋에 모델이 제대로 작동 X
 - 모델이 편향성을 가지고 있어 모든 학습 데이터를 다 고려해 주지 않는다.
- 보통 언더피팅이 되면 테스트 데이터셋에서도 잘 작동 X
- 언더피팅은 High Bias, Low Variance라 볼 수 있다.

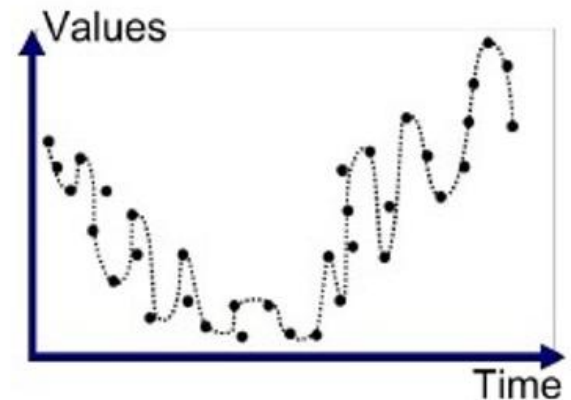
언더피팅 Underfitting



Underfitted



Good Fit/Robust



Overfitted

Bias와 Variance를 수식으로 보면

- 회귀 문제를 예로 들면 학습 샘플 데이터 $x_1, x_2, \dots, x_n$ 과 결과 $y_1, y_2, \dots, y_n$ 이 주어졌을 때, 가능한 한 실제 y_n 에 가장 가까운 추정치를 만들어내는 $\hat{y} = \hat{f}(x)$ 를 찾는 문제라고 볼 수 있다.
 - 최선이 될 수 있는 수많은 함수들이 있을 수 있다.
 - 어떤 함수가 제일 좋을지를 판단하기 위해서는 각 함수를 평가하는 척도 필요
 - ➔ Mean Square Error (MSE)

$$MSE = E[(y_i - \hat{y}_i)^2]$$

Bias와 Variance를 수식으로 보면

- 회귀 문제를 예로 들면 학습 샘플 데이터 $x_1, x_2, \dots, x_n$ 과 결과 $y_1, y_2, \dots, y_n$ 이 주어졌을 때, 가능한 한 실제 y_n 에 가장 가까운 추정치를 만들어내는 $\hat{y} = \hat{f}(x)$ 를 찾는 문제라고 볼 수 있다.

$$\begin{aligned}MSE &= E[(y - \hat{y})^2] \\&= E[(y - E[\hat{y}] + E[\hat{y}] - \hat{y})^2] \\&= E[(y - E[\hat{y}])^2] + E[(E[\hat{y}] - \hat{y})^2] + E[2(y - E[\hat{y}])(E[\hat{y}] - \hat{y})] \\&= E[(y - E[\hat{y}])^2] + E[(E[\hat{y}] - \hat{y})^2] + 2(y - E[\hat{y}])(E[\hat{y}] - E[\hat{y}]) \\&= (E[\hat{y} - y])^2 + E[\hat{y}^2] - E[\hat{y}]^2 \\&= Bias[\hat{y}]^2 + Variance[\hat{y}]\end{aligned}$$

오버피팅 vs. 언더피팅

- 기계학습의 목표는 오버피팅과 언더피팅을 모두 피하는 것!
- 즉, Low Bias, Low Variance를 달성하는 것이 목표
- 하지만 어려운 것은 보통 Bias와 Variance가 서로 Trade-Off 관계에 있기 때문이다.
 - * 이를 Bias-Variance Tradeoff 라고 부른다.

Performance Evaluation

성능 평가 방법 1

- 학습 데이터에 대한 성능이 떨어지면
(풀었던 연습문제를 다시 풀었을 때 성적이 좋지 않으면)
→ 언더피팅
- 학습 데이터에 대한 성능은 좋아도 오버피팅 가능

[단순한 방법]

오버피팅을 진단하기 위해 학습 데이터와 겹치지 않는

별도의 테스트 데이터에 대해 학습된 모델의 성능을 구해본다.

이때 성능이 떨어지면 → 오버피팅

학습 데이터, 테스트 데이터

전체 데이터를 학습 데이터와 테스트 데이터로 분리하고
학습 데이터를 이용해 모델을 학습 시킨 뒤 테스트 데이터로 평가

전체 데이터



학습 데이터

테스트 데이터

성능 평가 방법 2

- 학습 알고리즘마다 요구되는 학습 설정 존재
 - ➔ 같은 알고리즘 같은 데이터에 대해서도 설정에 따라 성능이 달라질 수 있음
- 학습 설정을 변수화 하여 하이퍼 파라미터라 부름
- 최적화 된 하이퍼 파라미터를 찾는 것도 중요!
 - ➔ 다른 하이퍼 파라미터 상황에서 성능 평가 결과를 비교
- 그런데 이 단계에서 테스트 데이터를 사용하게 되면? 반칙!
 - 오버피팅 여부(일반화 성능)를 평가할 수 없다.

학습 데이터, 검증 데이터, 테스트 데이터

전체 데이터를 학습 데이터, 검증 데이터, 테스트 데이터로 분리하고
검증 데이터에 대해 평가해 보면서 학습 데이터를 이용해 모델을 학습 시킨 뒤
테스트 데이터로 모델 성능을 최종 평가

전체 데이터



학습 데이터

검증 데이터

테스트 데이터

학습 및 최적의 하이퍼 파라미터 탐색

최종 평가

성능 평가 방법 2

하지만 하이퍼 파라미터가

학습 데이터 + 검증 데이터 한가지 조합에만 특화될 수도 있다.

➔ 이렇게 되면 테스트 데이터 성능이 잘 나오지 않을 것

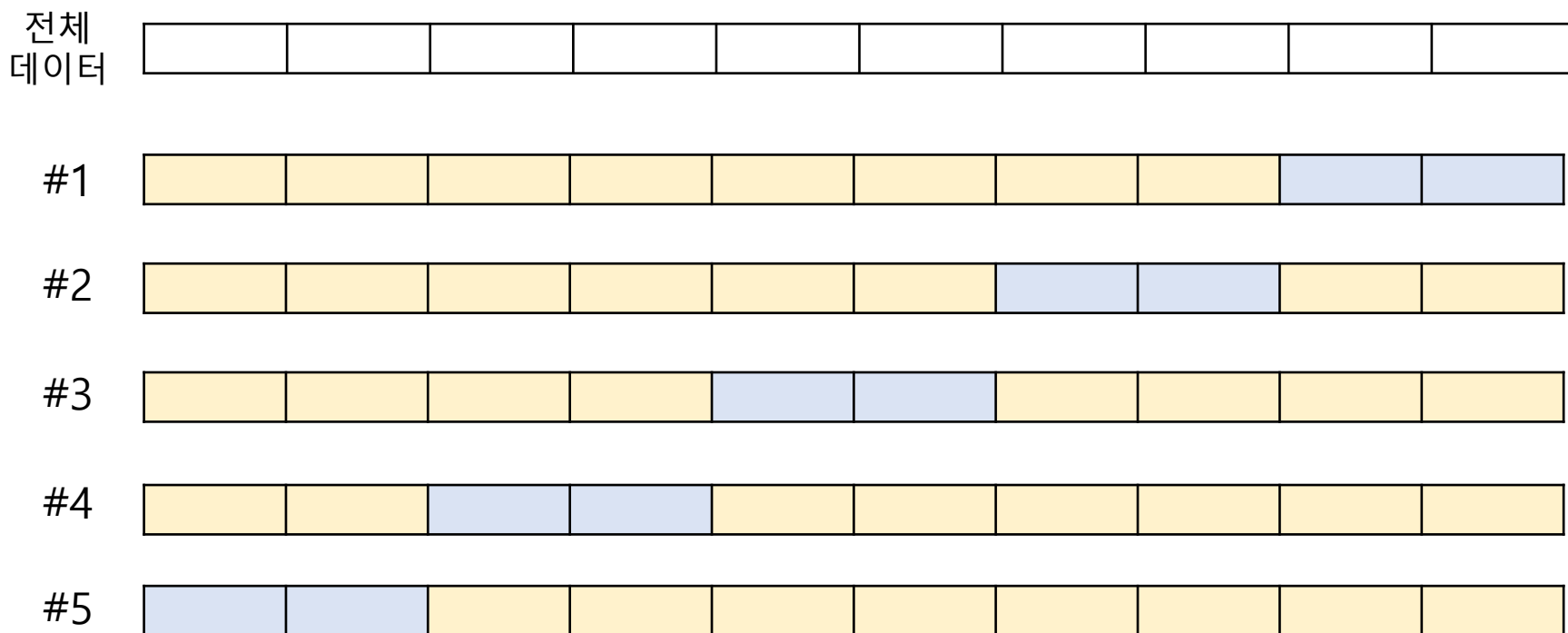
(물론! 데이터가 엄청 많다면 한가지 조합에서도 일반화 된 하이퍼 파라미터가 찾아질 것)

그래서!

일반적으로 Cross-Validation 방법을 사용

교차검증 Cross-Validation

- 데이터를 학습 데이터와 검증 데이터로 나누는 것을 여러 번 수행하여 학습-검증 데이터 조합을 여러 개 두고 각 조합 별 성능 평가를 평균해서 최종 평가하는 방법



교차검증 Cross-Validation

- Leave-One-Out Cross Validation (LOOCV)

- 전체 데이터에서 샘플 하나씩 돌아가면서 검증 데이터로 사용하고 나머지는 학습 데이터로 사용하는 방법
- 데이터가 n 개 있으면 n 번 실행 → 시간이 오래 걸림

- K-fold Cross Validation

- 전체 데이터를 k 개 그룹으로 나누고 한 그룹 씩 돌아가면서 검증 데이터로 사용하고 나머지 그룹들은 학습 데이터로 사용하는 방법
- 보통 k 는 5, 10을 많이 사용함

- K-Holdout Cross Validation

- k 번 성능을 평가하되 한번 평가할 때마다 전체 데이터에서 무작위로 학습 데이터와 검증 데이터를 만들어 사용 → 일부 데이터가 검증 데이터에 중복되거나 제외 될 수 있음

교차 검증을 통한 성능 평가

전체 데이터

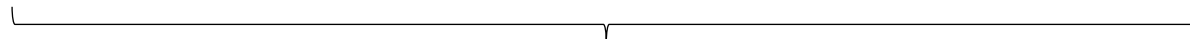


학습 데이터

검증 데이터

테스트 데이터

...



학습 및 최적의 하이퍼 파라미터 탐색

최종 평가

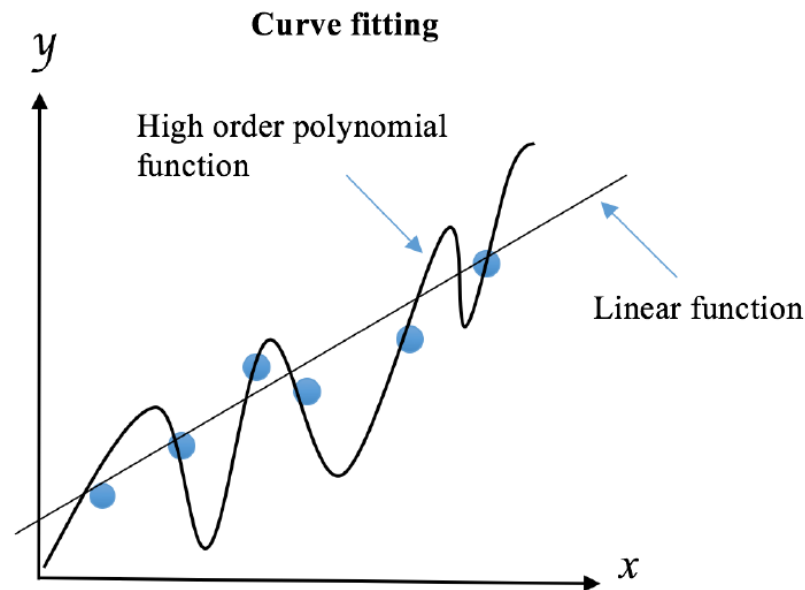
Avoiding Overfitting

학습 시 오버피팅 피하기

- Cross-Validation은 학습된 모델의 성능을 정확히 평가하여 오버피팅 여부를 판단하게 해준다.
- 학습 과정 자체에서 오버피팅을 미리 예방하기 위한 방법
 - 정규화 (Regularization)
 - 조기 중단 (Early Stopping)
 - 특성 선택 (Feature Selection)
 - 차원 축소 (Dimension Reduction)

정규화 Regularization

- 모델이 복잡할 수록 오버피팅에 빠지기 쉽다.
 - 모델이 복잡하면 표현할 수 있는 데이터 범위가 넓다 보니 학습 데이터에 딱 맞는 모델을 만들기 쉽다.
하지만 학습 데이터 너무 딱 맞는 모델은 일반화 된 모델은 아닐 수 있다.



정규화 Regularization

- 정규화는 학습 중에 모델이 복잡해질 수록 불이익을 받도록 하는 방법
 - 모델이 복잡해 졌을 때 오히려 성능을 더 안 좋게 되도록 원래 에러 함수에 정규화 변수를 추가한 것이다.
 - 예) 고차원 다항 모델에 패널티를 줘서 다항 모델의 고차항이 줄 수 있는 영향력을 감소시킨다.
 - 정규화 파라미터 값은 잘 조절되어야 한다.
 - 너무 작으면 영향이 없어 오버피팅 문제를 야기하고
 - 너무 크면 학습이 제대로 안되는 언더피팅 문제가 야기될 수 있다.

조기중단 Early Stopping

- 학습을 조기에 중단시키는 방법
- 예시
 - 모델 학습을 위해 소요되는 시간에 제약을 두는 방법
 - 학습 중단 조건을 설정하는 방법
 - 예) 검증 데이터 성능이 XX% 이상 되면 중단

특성 Feature

- 머신러닝에서 데이터는 주로 행렬 같은 그리드로 표현
 - 각 열들은 변수
- 머신러닝 목적에 따라 이 중 하나의 변수가 '레이블' 예측하려는 값이 된다.
- 레이블을 제외한 변수들을 **특성(feature)**라고 한다.
 - 특성들이 이루는 공간을 **차원(Dimension)**이라 하기도 한다.
- 즉, 머신러닝은 특성들을 가지고 레이블을 예측하는 문제

성별	나이	플레이시간	플레이횟수	게임승률
M	16	35	110	53.5
F	20	50	150	60.7
M	12	20	13	30.4

차원의 저주 Curse of Dimensionality

- 머신러닝에 있어서 특성이 많으면 많을 수록 좋을까?
 - 꼭 그런 것은 아니다.
 - 특성이 늘어날 수록 특성들이 가질 수 있는 값의 조합이 엄청나게 늘어난다.
 - 이 조합들을 모두 보려면 복잡한 모델과 많은 데이터가 필요할 것이다.
 - 오버피팅의 위험도 증가
 - 경우의 수가 늘어나는 것에 비해 실제 값들이 분포하는 곳은 많지 않을 수 있다.
 - 따라서 적당한 (적은) 차원에서 문제를 풀 수 있으면 좋다.

특성 선택 Feature Selection

- 데이터 내 존재하는 모든 특성들 중 일부만 선택해 사용하는 방법
- 후진 방법
 - 특성을 처음에는 전체 다 사용하고 반복 시행 단계마다 특성을 제거해 나가는 방법
- 전진 방법
 - 가장 최소한의 특성 집합으로 시작해 반복 시행 단계마다 특성을 늘려 나가는 방법

차원 축소 Dimension Reduction

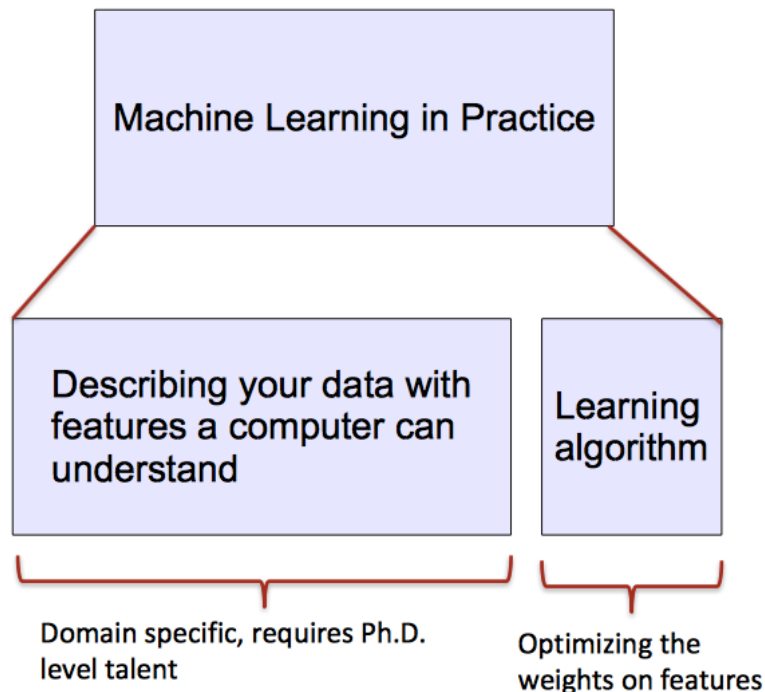
- 고차원 데이터를 저차원 공간의 데이터로 변환하는 방법
 - 예) Singular Vector Decomposition → Principle Component Analysis
- 특성 선택은 처음 데이터의 특성들 값이 유지는 되지만
차원 축소는 차원을 변환하기 때문에 특성의 종류와 값 자체가 바뀐다.
- 변환 하면서 손실되는 정보가 있을 수 있는데 이를 최소한으로 유지

(특성 파악하기 위한 방법) 특징 추출 방법들

Machine Learning vs. Deep Learning

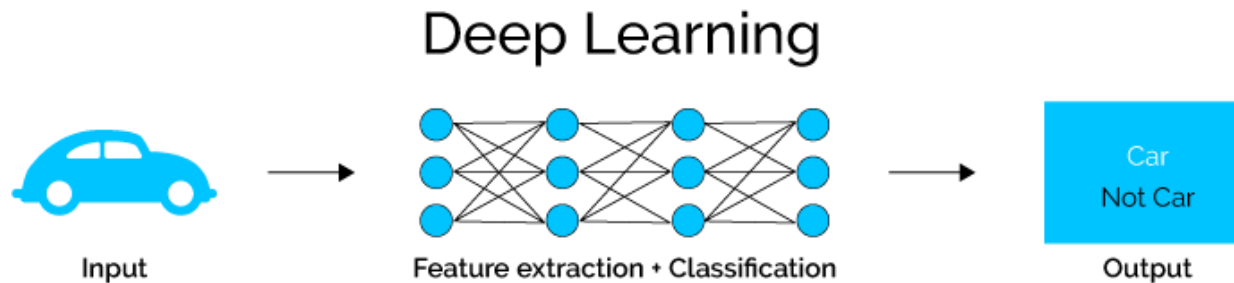
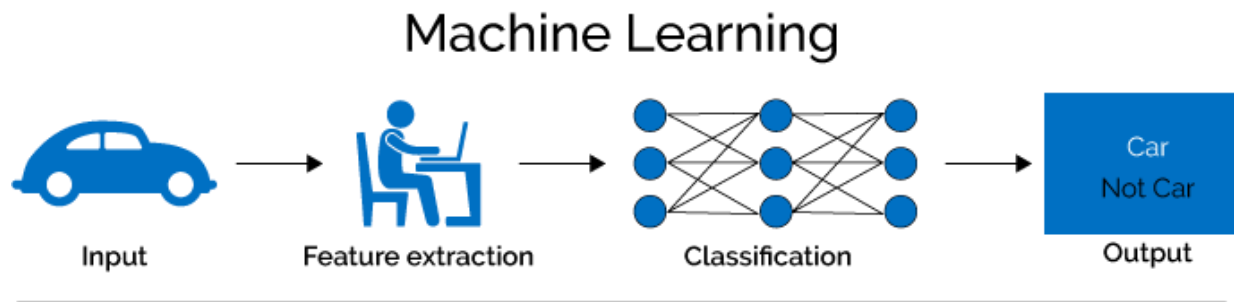
ML vs. DL

- Most machine learning methods work well
 - Due to human-designed representations and input features
- ML (somewhat) becomes optimizing weights to best make a final prediction



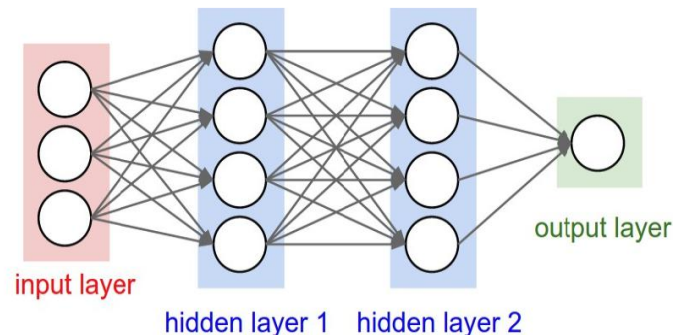
ML vs. DL

- Deep learning
 - A part of a broader family of machine learning methods based on learning data representations, as opposed to task-specific algorithm



Deep Learning

- Uses a cascade of multiple layers of nonlinear processing units for feature extraction and transformation
 - Each successive layer uses the output from the previous layer as input
- Learns in supervised (e.g., classification) and/or unsupervised (e.g., pattern analysis) manners
- Learns multiple levels of representations that correspond to different levels of abstraction; the levels form a hierarchy of concepts

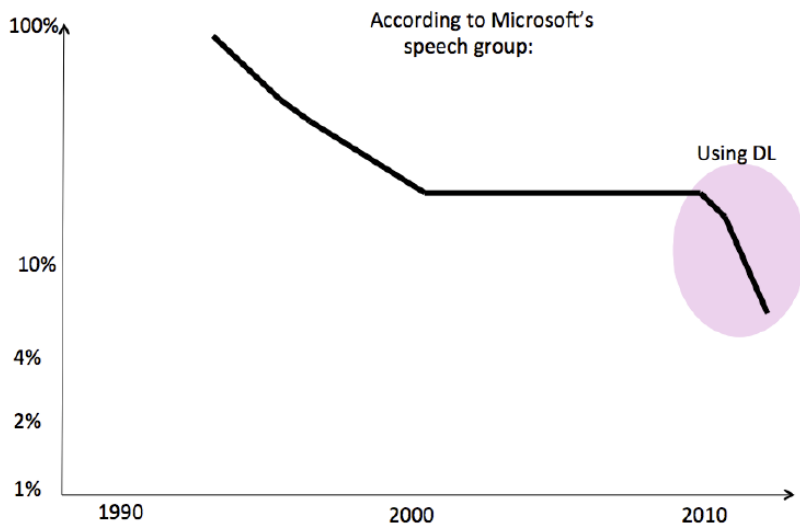


Deep Learning

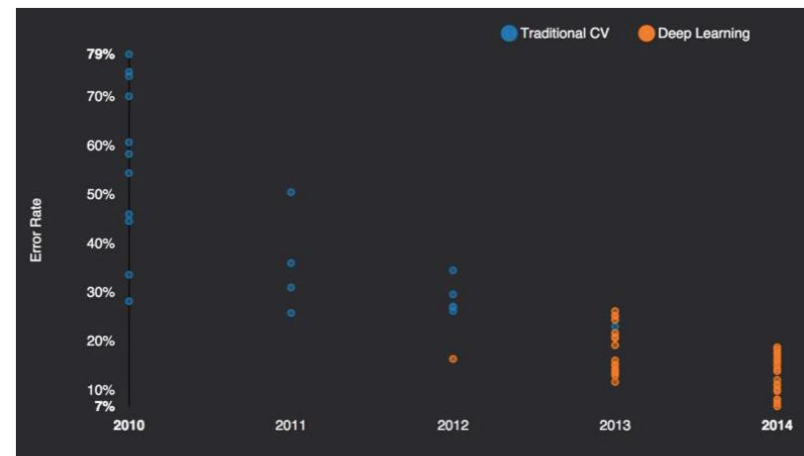
- ML: manually designed features are often over-specified, incomplete, and take a long time to design and validate
- DL provides a very flexible, (almost?) universal, learnable framework for representing visual and linguistic information
- DL utilizes a large amount of training data
 - Pros and cons?
- From 2010, DL has started to outperform other ML techniques, first in speech and vision, then NLP

Deep Learning

- Big improvements in recent years in NLP and computer vision!
 - DL + Big Data (e.g., ImageNet) + Computing Power



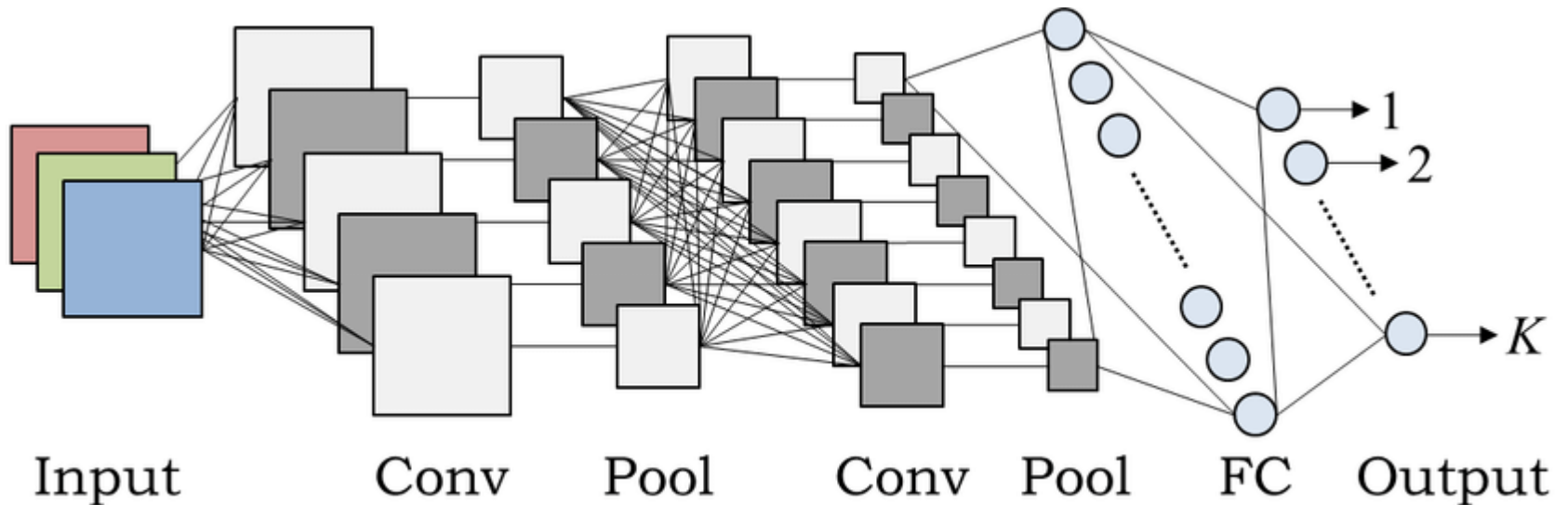
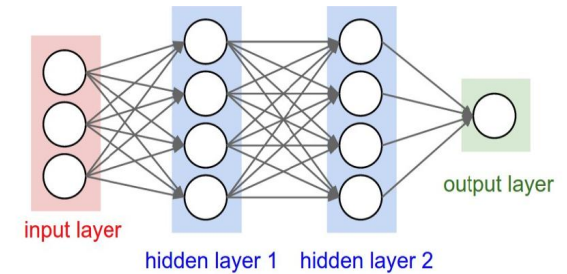
Deep Learning in Speech Recognition



ImageNet: The "computer vision World Cup"

CNN

- Convolutional Neural Network (CNN)
 - A neural network that uses “convolution”
 - Convolution? -- filter

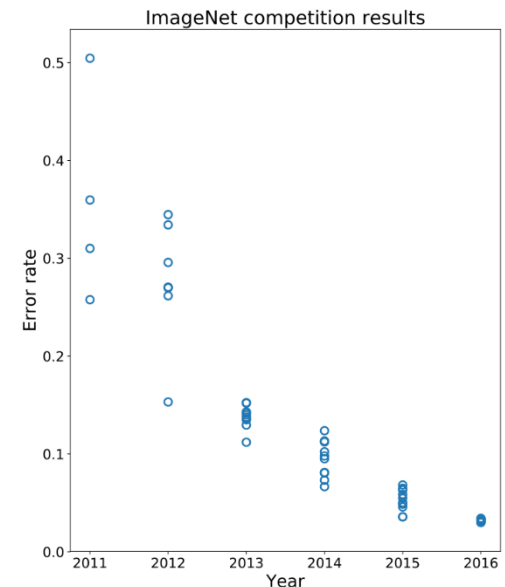


예: 이미지에서 어떤 특징이 얼마나 있는지 구하는 과정

예: 이미지의 뒤틀림이나 크기변화에 따른 왜곡 영향 축소

ImageNet

- A large visual database designed for use in visual object recognition research
 - Originally, over 14 million URLs of images have been hand-annotated by ImageNet, containing over 20 thousand categories
- ImageNet Challenge (2010~) for 1000 image categories
 - Deep Learning achieves dramatic performance!
 - In 2017, 29/38 teams achieved less than 5% error

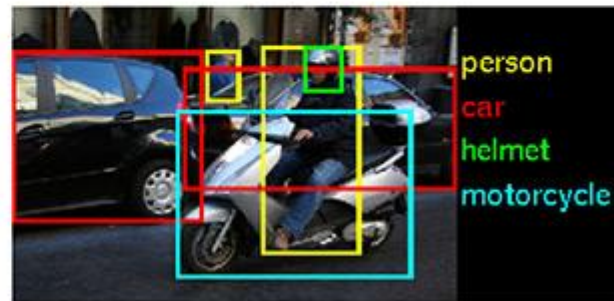
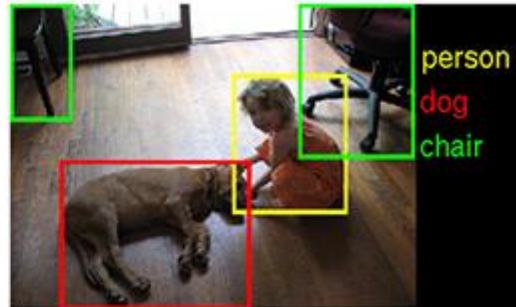
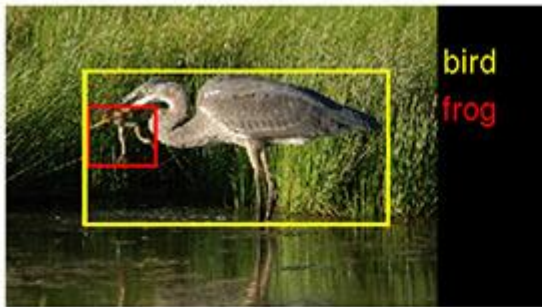


CNN Models

- AlexNet (2012) – (almost) first CNN, 15.4%
- ZFNet (2013) – 11.2%
- VGGNet (2014) – 7.3%
- GoogLeNet (2014) – 6.7%
- ResNet (2015) – 3.57%
- CUIImage (2016) – 2.99%
- ...

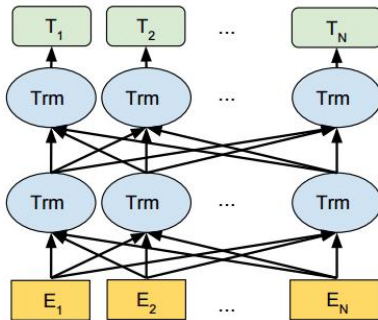
CNN Models

- Object Detection



Natural Language Processing

- BERT models



Natural Language Processing

- Tacotron, TTS



RNN

- Recurrent Neural Network (RNN)
 - Sequence learning
 - Long-Term dependency problem
- LSTM (Long Short Term Memory) networks
 - Solving the long-term dependency problem
 - Widely used

RL

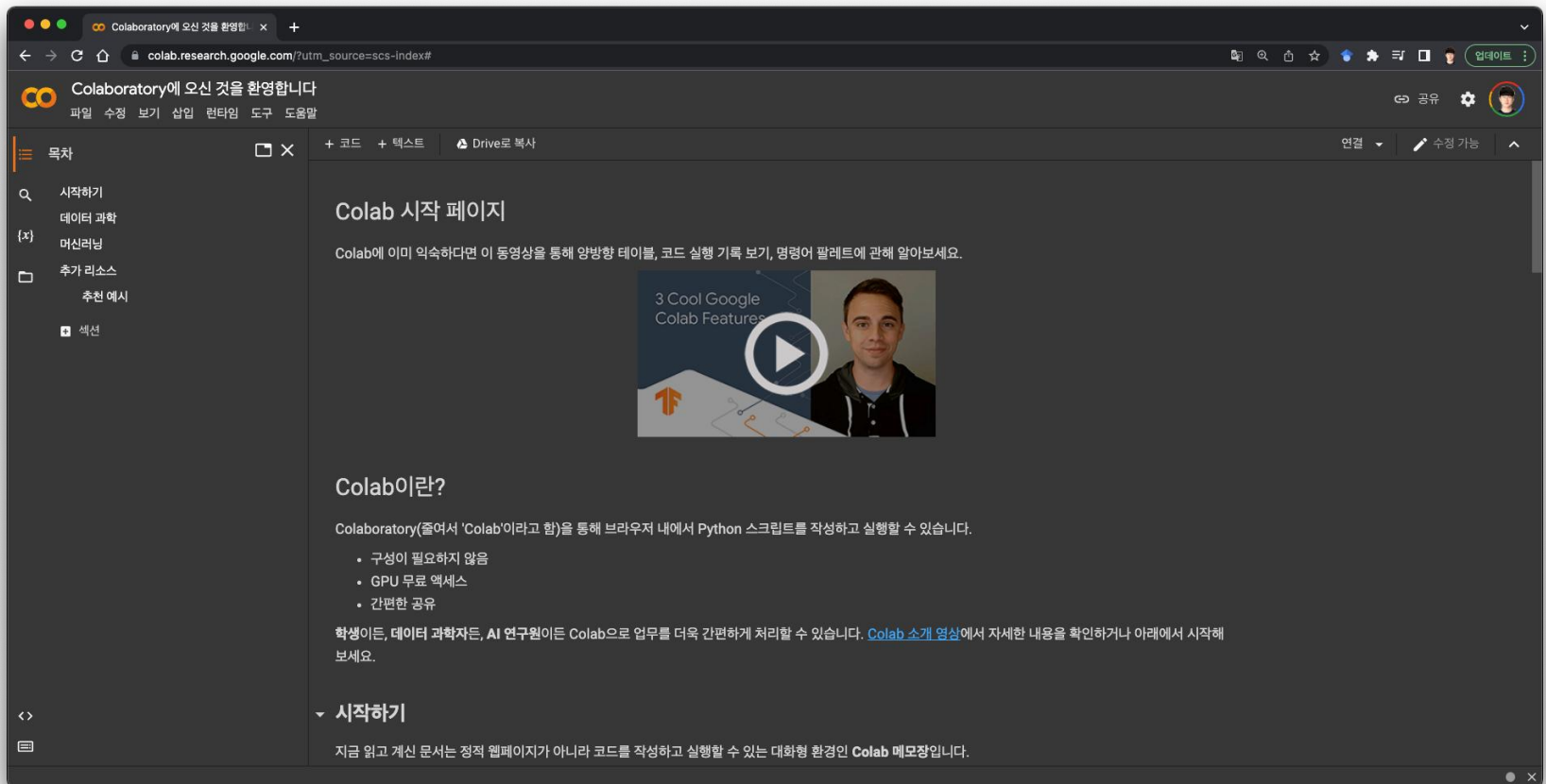
- RL (Reinforcement Learning)
 - A problem of getting an agent to act in the world so as to maximize its rewards
- AlphaGo!



Lab 1-1: Colab & Python

How to use Google Colab

- Google Colab: <https://colab.research.google.com/>



Lab Materials

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Lab 1-2: Numpy & Pandas

Lab Materials

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.